

Printing the Maze Data Structure

To represent the maze as a data structure, the Python program uses a dictionary, and the JavaScript program uses an object. Within this structure, the keys are lists or arrays of two integers for the x- and y-coordinates, while the value is either the `WALL` or `EMPTY` single-character strings. Thus, we can access the wall or empty hallway space at the coordinates x, y in the maze as `maze[(x, y)]` in Python code and as `maze[[x, y]]` in JavaScript code.

The Python code for `printMaze()` starts as follows:

Python

```
def printMaze(maze, markX=None, markY=None):
    """Displays the maze data structure in the maze argument. The
    markX and markY arguments are coordinates of the current
    '@' location of the algorithm as it generates the maze."""

    for y in range(HEIGHT):
        for x in range(WIDTH):
```

The JavaScript code for `printMaze()` starts as follows:

JavaScript

```
function printMaze(maze, markX, markY) {
    // Displays the maze data structure in the maze argument. The
    // markX and markY arguments are coordinates of the current
    // '@' location of the algorithm as it generates the maze.
    document.write('<code>');
    for (let y = 0; y < HEIGHT; y++) {
        for (let x = 0; x < WIDTH; x++) {
```

The `printMaze()` function prints the maze data structure it's passed as the `maze` parameter on the screen. Optionally, if `markX` and `markY` integer arguments are passed, the `MARK` constant (which we set to `@`) appears at these x, y coordinates in the printed maze. To make sure the maze is printed in a monospace font, the JavaScript version writes the HTML tag `<code>` before